

# Playwright Ecommerce Quality Framework

## Technical study workbook and interview preparation

Prepared for Stefan Kajchevski. Repository 1:  
`stefank3/playwright-ecommerce-quality-framework`.

Implementation baseline: `3193e84826dd3d01af1bf43295e11e7988acc605`, including merged PR #5. This workbook explains existing behavior. It changes no scenarios, fixtures, selectors, network policy or CI execution.

## How to study

Read lessons 1-10 for the system map, 11-23 for execution boundaries, and 24-30 for extension, diagnosis and interview practice. Each lesson connects a concrete implementation to a design reason and a question you should be able to answer aloud. Use the contents page or PDF bookmarks to navigate.

Keep the repository open beside this book. Paths are relative to its root.  
`docs/code-walkthrough.md` is the complete maintained-file index; `docs/interview-guide.md` contains timed interview scripts. Existing architecture, standards, security and live-testing documents remain the operational references.

## Evidence and execution boundary

The baseline deterministic gate contains 49 tests. The separate live suite contains four UI and five API scenarios. Historical live evidence is dated 2026-09-24; no live run is needed to study this material. Commands describing live execution are reference material and require separate authorization. The interview demonstration uses only deterministic commands.

## Canonical source

This Markdown workbook is canonical. `npm run docs:pdf` generates its sibling PDF using `scripts/docs_pdf.py` and pinned `guide/requirements.txt`. The PDF must never be manually edited. The generation contract, installation steps and review procedure are in `docs/extending.md`.

# Contents

|                                                 |    |
|-------------------------------------------------|----|
| 01. Purpose and problem statement               | 3  |
| 02. Repository tour                             | 4  |
| 03. Architecture and modular-monolith rationale | 5  |
| 04. Why Playwright                              | 6  |
| 05. Why strict TypeScript                       | 7  |
| 06. Why Zod runtime validation                  | 8  |
| 07. Why fixture-based deterministic testing     | 9  |
| 08. Deterministic execution lane                | 10 |
| 09. Live execution lane                         | 11 |
| 10. Playwright projects and configuration       | 12 |
| 11. Test fixtures and dependency injection      | 13 |
| 12. Page Object Model in this repository        | 14 |
| 13. Typed API contracts and client flow         | 15 |
| 14. UI scenario anatomy                         | 16 |
| 15. API scenario anatomy                        | 17 |
| 16. Network-denial preload                      | 18 |
| 17. Browser request allowlisting                | 19 |
| 18. Budgets, spacing, locks and time limits     | 20 |
| 19. Error sanitization                          | 21 |
| 20. Screenshots, traces and reports             | 22 |
| 21. CI and the automatic gate                   | 23 |
| 22. Headless and headed execution               | 24 |
| 23. Scripts and command wrappers                | 25 |
| 24. How to add a UI page object                 | 26 |
| 25. How to add an API contract                  | 27 |
| 26. How to add deterministic scenarios          | 28 |
| 27. How to add live scenarios safely            | 29 |
| 28. Debugging and troubleshooting               | 30 |
| 29. Framework limitations and evidence          | 31 |
| 30. Interview exercises and answers             | 32 |

# 01. Purpose and problem statement

## The problem this repository addresses

A test suite can fail because the framework is wrong, because a public site changed, or because the execution environment is unavailable. Mixing those causes into one required gate makes results hard to interpret. This repository separates controlled framework evidence from dated live observations.

There is no ecommerce application here. The maintained product is a test framework: orchestration, test data, page objects, API boundaries, safety controls and reports. The synthetic UI is minimal hand-authored markup, not a local replica of the external site. The controlled API is an injected response, not a running server.

| Evidence           | What it can support                                                                    |
|--------------------|----------------------------------------------------------------------------------------|
| Deterministic pass | Maintained scenarios and boundaries work with controlled inputs on the tested runtime. |
| Live pass          | The selected external interactions met expectations at a recorded time.                |
| Neither alone      | Production impact, comprehensive correctness, accessibility or broad browser support.  |

## A concrete example

The synthetic catalog has one item called Synthetic Blue Shirt. The live quantity scenario checks Blue Top at 500 rupees with quantity three. Their similarity teaches the same testing concepts, but their inputs and evidence are deliberately different. A synthetic pass cannot establish that the live product still costs 500.

## Why this design?

The required gate remains useful when a public practice site is slow or unavailable. Separately authorized live testing can then expose integration drift without making every code change depend on external availability. The distinction also makes interview claims easier to defend: explain exactly which observation supports each claim.

## Check your understanding

If all 49 deterministic tests pass, may you say the provider cart works today? **Answer:** no. You may describe the framework checks that passed and separately cite dated live evidence. Deterministic testing controls inputs; it does not manufacture knowledge about an uncontacted service.

## 02. Repository tour

### Find behavior by responsibility

Start with `package.json` to learn supported commands. Read the selected Playwright configuration next, then the scenario and its fixture. Follow an imported object only when you need to explain a specific boundary. This is more useful than memorizing the directory tree alphabetically.

| Location                                   | Responsibility                                                                   |
|--------------------------------------------|----------------------------------------------------------------------------------|
| <code>tests/</code>                        | Scenario intent and assertions; deterministic and live directories are disjoint. |
| <code>src/fixtures/</code>                 | Construct collaborators, scope state, install guards and audit teardown.         |
| <code>src/ui/</code>                       | Page interactions and meaningful locators.                                       |
| <code>src/api/</code>                      | Injected client, fixed live transport and pure contracts.                        |
| <code>src/config/</code>                   | Parse settings, validate outputs and classify/sanitize failures.                 |
| <code>scripts/</code>                      | Preflight, guarded execution, build and repository/PDF tooling.                  |
| <code>docs/</code> and <code>guide/</code> | Operational references, review history and study material.                       |

### Maintained inputs versus generated outputs

The lockfile and configuration are maintained inputs. `dist/`, `test-results/` and `playwright-report/` are ignored outputs. Installed dependencies and the maintainer's `.runtime/` are also ignored. The study PDF is an explicit exception: it is maintained generated evidence, reproducible from its canonical Markdown.

The curated `docs/assets/live-report.png` represents the first historical live run. It is not a current report and does not contain a screenshot of the target page. Its provenance matters more than its appearance.

### Why this design?

Separating files by responsibility limits the search area when a failure appears. A wrong expected total belongs in scenario reasoning; a changed provider locator belongs in the page object; invalid JSON belongs at a contract/transport boundary. That does not mean every failure has an obvious cause, but it gives diagnosis a starting point.

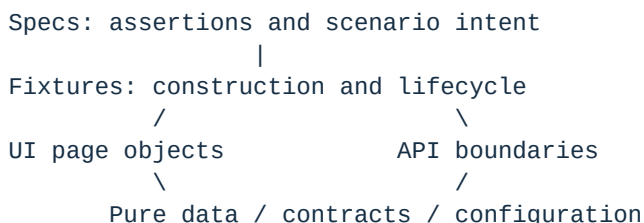
### Check your understanding

Where would you look when an API body is accepted despite a malformed ID? **Answer:** the pure schema and its deterministic rejection tests, then the call site that should invoke the parser. Do not begin by changing a browser selector.

## 03. Architecture and modular-monolith rationale

### One repository, focused boundaries

The architecture is a test-focused modular monolith. It uses one toolchain and in-process composition rather than separately deployed services. This is proportionate to one framework and a small approved scenario set. There is no message bus, database or service registry to operate.



Scripts and Playwright config govern execution.  
Reporting observes results and safe diagnostics.

The diagram summarizes allowed collaboration; UI objects do not call API clients. Pure contracts can import Zod and safe errors, but never fixtures or transport. Data builders may import contract types without gaining a runtime HTTP dependency. Specs are not imported by support modules.

### Why this design?

The smallest useful abstraction is easier to explain, test and review. `ProductClient` accepts a function rather than requiring a provider registry. Live page objects are small products/detail/cart objects, not subclasses of a giant base page. Playwright supplies the lifecycle system, so a custom dependency-injection container would duplicate existing behavior.

This choice has costs. The framework is intentionally specialized, and a new provider or lifecycle would require an approved design change. It is not a configurable commercial test platform. Specialization is useful when it keeps external effects bounded and interfaces truthful.

### Enforcement and limits

`check-repository.mjs` checks selected local-import directions, while TypeScript and lint catch other code errors. A source-pattern check is not a complete semantic dependency analysis. Review is still needed to detect indirect coupling or inappropriate use of an otherwise allowed dependency.

### Check your understanding

Should a page object import a spec helper to reuse an assertion? **Answer:** no. That reverses the dependency direction. Keep scenario expectations in tests and expose a meaningful locator or observation through the page object.

## 04. Why Playwright

### The capabilities actually used

Playwright provides the runner, Chromium automation, locators, web-first assertions, request contexts, fixtures and HTML reporting. Using these together reduces custom infrastructure. The repository pins @playwright/test at 1.63.0; its current implemented browser coverage is Chromium only.

Consider the controlled UI assertion:

```
await expect(catalog.products().getByRole('listitem')).toHaveCount(1);
```

This assertion checks an observable outcome and waits within the configured assertion timeout. It avoids a fixed sleep that could be too short on a slow machine or unnecessarily long on a fast one. The page object returns the locator; the spec chooses the expectation.

### Runner versus browser

Playwright can run tests that use no page. Many deterministic boundary tests are ordinary TypeScript checks under the Playwright runner. Live API operations use APIRequestContext, not browser navigation. In the current shared fixture modules, however, automatic guards request a browser context even for API scenarios. Tool capability and this repository's fixture graph are different questions.

### Why this design?

A single runner gives consistent discovery, timeouts and results across UI and API scenarios. Native fixtures already express per-test isolation and worker-shared dependencies. Built-in reports avoid building a separate results service. The small custom live reporter exists for sanitization and numeric evidence, not as a replacement test runner.

### Trade-off

Pinning the runner makes behavior reviewable but requires deliberate dependency maintenance. Browser binaries must be installed separately. A headless Chromium pass does not certify Firefox, WebKit or the visual behavior of blocked third-party assets.

### Check your understanding

Does choosing Playwright establish cross-browser coverage? **Answer:** no. The configured projects and executed evidence establish coverage. Both current projects select Chromium; additional browsers remain deferred.

## 05. Why strict TypeScript

### Make uncertainty visible

`tsconfig.json` enables strict checking, unchecked-index awareness, exact optional properties and checked overrides, alongside unused-local/parameter checks. NodeNext module resolution matches the ESM package. Typecheck covers support code, specs and Playwright configuration; the build emits only support code and declarations.

The live adapter exposes a small operation union:

```
type ApiOperation =  
  'products' | 'brands' | 'search' | 'unsupported' | 'malformed';
```

This gives a reviewer a finite set of intended operations. A spec cannot accidentally pass an arbitrary endpoint through the typed public interface. The adapter still checks origins and redirects at runtime because a type is not a network security boundary.

### Compile-time and runtime are complementary

An external response enters as unknown. Declaring it as `ProductList` with a cast would only tell the compiler to trust the author. It would not examine JSON. Zod supplies that runtime examination, after which inferred types help downstream code.

Type-aware ESLint complements the compiler. `no-floating-promises` rejects unawaited Playwright assertions, including promises discarded with `void`; `await-thenable` rejects misleading awaits. A narrow commented exception permits Playwright's required empty destructured fixture parameter.

### Why this design?

The main benefit is explicit contracts between collaborators. It becomes easier to refactor an operation name, fixture dependency or return value without relying entirely on live execution to discover mistakes. Strictness does not replace meaningful tests or prove the provider's behavior.

### Check your understanding

Why keep body: unknown in the transport response? **Answer:** receiving bytes and decoding JSON do not establish a domain contract. The parser must validate the consumed fields before the body becomes trusted typed data.

## 06. Why Zod runtime validation

### Validate the fields you consume

`src/api/contracts/products.ts` requires positive integer IDs, nonempty names up to 120 characters and prices matching `Rs. <digits>`. The envelope requires provider `responseCode`: 200 and between one and 100 products. Failed parsing becomes a fixed `FrameworkError` message without payload values.

```
const productSchema = z.object({
  id: z.number().int().positive(),
  name: z.string().min(1).max(120),
  price: z.string().regex(/^Rs\.\s\d+$/),
});
```

The schemas accept legitimate additive fields and discard them from parsed output. For example, an extra provider brand field need not break a consumer that reads only ID, name and price. Consumed fields remain strict: a string ID is not silently accepted as a numeric ID.

### Schema rules versus scenario rules

The brand parser validates shape, size and field types. The live brands spec separately asserts unique IDs. The rejection parser accepts provider 400 or 405 and a bounded message; each negative scenario separately checks the exact expected code and message. This separation keeps reusable contracts distinct from specific business expectations.

### Why this design?

Failing on every harmless provider extension makes a suite fragile. Accepting malformed consumed fields makes it misleading. A focused consumer contract balances these concerns without claiming full provider-schema certification.

Configuration is also an input boundary. The deterministic config uses Zod coercion and limits for timeout, while explicit key filtering rejects unknown framework-owned keys. Errors never echo rejected environment values.

### Check your understanding

Will an empty product array pass because it is an array? **Answer:** no. The contract has a minimum length of one. Conversely, a valid array does not prove a particular product exists; that belongs to the scenario assertion.



## 07. Why fixture-based deterministic testing

### Own the inputs

`buildProducts()` returns a fresh synthetic object on every call. The controlled document is short static HTML with an accessible Products list, one heading and one price. Neither is a raw capture from the provider. A test can therefore know exactly which input produced its result.

```
new ProductClient(async () => ({
  status: 200,
  body: buildProducts(),
}));
```

This is the composition used by the deterministic API fixture. It exercises the consumer's status and parsing path without opening an HTTP connection. It does not exercise a server or prove provider integration.

### Lifecycle matters as much as data

Playwright fixtures construct the dependencies requested by the test and bracket their use through `await use(...)`. The automatic network guard is installed before page creation and audited afterward. Fresh data and fresh contexts prevent one scenario from depending on a previous scenario's mutations.

A deterministic test explicitly empties one builder result and checks that a second call still has one product. That verifies the isolation property directly, rather than assuming a function name implies freshness.

### Why this design?

The framework can test malformed payloads, rejected configuration, denied requests and transport failures without requiring those failures to occur at a public service. Failures are easier to reproduce because the test owns the triggering input. The cost is a deliberate evidence limit: synthetic content may not resemble every future provider behavior.

### Check your understanding

Why not share one mutable product object across tests? **Answer:** a mutation could change another test's input and make outcome depend on execution order. A small fresh builder is simpler than cleanup rules for shared mutable data.

## 08. Deterministic execution lane

### Follow the wrapper first

`npm test` and `npm run test:deterministic` both call `scripts/run-tests.mjs`. It parses the environment and validates artifact roots before spawning Playwright. Accepted arguments are `--list`, `--debug` and `--grep <pattern>`. Arbitrary config and runner overrides are rejected.

```
run-tests.mjs
-> readConfig + checkTestOutputs
-> NODE_OPTIONS imports deny-network.mjs
-> playwright.config.ts checks guard marker
-> worker inherits denial preload
-> tests/deterministic only
```

The default timeout is 30 seconds, configurable from 1000 to 30000 through `QE_TIMEOUT_MS`. Assertions use five seconds. One worker and zero retries keep the baseline simple. `forbidOnly` rejects accidentally focused tests.

### What the 49 tests contain

There are two controlled scenario tests, fifteen general boundary cases, fourteen live-boundary checks, four fake live-transport checks and fourteen primitive/preload checks. A test filename containing “live” does not mean that the test contacts the provider. Its imports and injected collaborators determine what it actually does.

### Why this design?

The preload starts before discovery and remains present in workers. Direct unguarded Playwright discovery fails at the configuration check. A separate child-process test proves that rejection. Browser offline settings and routes cover the browser side; Node patches cover the tested Node transport primitives.

### Try this locally

```
npm run test:list
npm test -- --grep "read-only"
```

Use the supported runtime with no live variables. The first command discovers tests; the second executes the two controlled scenarios. Neither selects live specs. These commands are sufficient for an interview demo; a live run adds no necessary teaching value.

### Check your understanding

What happens if `QE_LIVE` is set during `npm test`? **Answer:** deterministic configuration rejects the unknown framework key. It does not switch to the live lane.

## 09. Live execution lane

### Separate intent and effects

Live execution has its own wrapper, configuration and test directory. `readLiveConfig` requires exact `QE_LIVE=true` and an exact approved HTTPS origin in `QE_BASE_URL`; other `QE_` keys are rejected. The accepted hosts are the bare and `www` Automation Exercise hosts. This workbook does not authorize contacting either.

The wrapper validates outputs and arguments, creates a local lock, injects a marker preload and selects the full suite or UI/API subset. Configuration rejects direct invocation without the marker. Listing still validates opt-in, but it does not dispatch target requests.

### Scenarios actually implemented

| UI: four scenarios                    | API: five scenarios                      |
|---------------------------------------|------------------------------------------|
| Search for Blue Top                   | List products                            |
| Add two products and check cart cells | List brands and unique IDs               |
| Add quantity three and check total    | Search fixed product term                |
| Remove product 1 from the session     | Unsupported method and missing parameter |

No scenario registers an account, posts a review, subscribes, checks out, pays or creates an order. Cart changes are isolated session effects, not a promise that every GET is inherently read-only.

### Why this design?

Explicit lane selection prevents a routine validation command from becoming an external test. Fixed targets and operations reduce accidental scope expansion. Current live evidence is historical and dated; the provider may change products, prices, markup or availability later.

### Failure interpretation

A live failure stops further scenarios. Report the failed and unexecuted cases honestly rather than counting unexecuted work as success. A focused rerun needs a concrete framework correction and appropriate authorization; repeated runs are not a substitute for diagnosis.

### Check your understanding

Does setting the correct environment variables grant permission? **Answer:** no. It satisfies a technical gate. User authorization and the documented policy check are separate operational requirements.

# 10. Playwright projects and configuration

## Two projects with disjoint discovery

The deterministic config defines `deterministic-chromium`; the live config defines `live-chromium`. Each selects its own test directory. Project names communicate intent, but directory selection and wrapper checks enforce the split.

| Setting           | Deterministic             | Live                                 |
|-------------------|---------------------------|--------------------------------------|
| Workers / retries | 1 / 0                     | 1 / 0                                |
| Test timeout      | 30 seconds by default     | 60 seconds                           |
| Assertion timeout | 5 seconds                 | 10 seconds                           |
| Global timeout    | No explicit config cap    | 300 seconds                          |
| Capture           | Failure trace/screenshots | Trace/screenshots/video off          |
| Browser           | Offline Chromium          | Offline Chromium with routed fetches |

Live actions have 15-second timeouts and navigations 45 seconds. HTTP adapter and routed fetch calls use 15 seconds. The wrapper adds a 315-second process timeout outside the runner's five-minute cap. These limits serve different scopes; do not add them together into an expected duration.

## Outputs and reporters

Each lane writes to separate report and result subdirectories. Running deterministic validation therefore does not overwrite the historical live report. The live sanitizing reporter is configured before built-in list and HTML reporters. `preserveOutput: 'never'` and disabled raw capture further reduce retained live evidence.

## Why this design?

Disjoint configuration makes the safe default visible. The common Chromium/offline settings do not imply identical transport behavior: deterministic routes only fulfill synthetic markup, whereas approved live routes deliberately fetch through a controlled HTTP path.

## Check your understanding

Does one project named Chromium mean every test opens a page? **Answer:** no. Fixtures are dependency-driven. A test may only use pure functions, request transport or a context. In shared scenario fixtures, the automatic audit creates a context even when the API test requests no page.

# 11. Test fixtures and dependency injection

## Construction belongs at the edge

The deterministic fixture extends Playwright with `catalog`, `productApi` and `automatic networkGuard`. The live fixture adds `products`, `cart`, `detail`, `api` and `automatic audit`, with worker-scoped `budget` and `cache`. A test declares the collaborators it needs in its argument rather than constructing them in each scenario.

```
Test requests detail and cart
  audit -> context + budget + cache
  detail -> page -> context
  cart   -> same page
  test actions and assertions
  audit + attachment + context cleanup
```

The shared page matters for cart scenarios: `detail` and `cart` views see the same isolated session. A different test receives a fresh context. Only static asset cache and traffic budget are shared at worker scope; the cache excludes session HTML and cookies.

## A minimal injection boundary

`ProductClient` receives a function implementing one fixed endpoint. `LiveHttpTransport` receives a narrow HTTP requester, base origin and dispatcher. Deterministic transport tests implement that surface with in-memory responses. Injection is valuable because it separates effects from decision logic, not because a DI library is present.

## Why this design?

Fixtures centralize ordering and cleanup. The test can focus on intent, and supporting tests can replace the effectful boundary. Worker sharing is narrowly justified by whole-run throttling and reusable static assets. Sharing mutable carts or products would undermine isolation.

## Important nuance

Automatic fixture dependencies run even when a scenario does not explicitly request them. Both scenario fixture sets request context through an automatic guard/audit. Their API operations are independent of browser pages, but their complete fixture lifecycle is not browser-free. Removing that overhead is future implementation work, not a documentation correction.

## Check your understanding

Can the API test avoid context setup merely by omitting page? **Answer:** not with the current shared fixture graph. The automatic fixture still depends on context.

## 12. Page Object Model in this repository

### Small objects, visible expectations

The controlled `ProductListPage` only opens the synthetic listing and exposes the accessible `Products` list. Live objects represent the product listing, one product detail flow and the session cart. Constructors bind a page without navigation or assertions.

```
const cells = cart.cells(1);
await expect(cells.quantity).toHaveText('3');
await expect(cells.total).toHaveText('Rs. 1500');
```

This excerpt shows the boundary: the cart object returns meaningful cell locators, and the scenario supplies expected values. Hiding all assertions inside a page object would make the scenario less explicit and could encourage unrelated tests to inherit the same expectation.

### Locator reasoning

Use roles, placeholders, headings and domain relationships where available. The live search input has a placeholder, but its submit button lacks a useful accessible name, so the object uses `#submit_search`. Product cards are scoped by exact product name inside `.productinfo` to avoid duplicate hover presentation. Cart rows use product IDs, not positional selectors.

CSS is therefore a deliberate fallback, not evidence that accessibility was tested. A selector working today does not establish stable provider markup. Failures can still require review after a DOM change.

### Why this design?

Focused objects keep provider-specific interactions out of specs while avoiding a generic page hierarchy. There is no abstract base class, universal click wrapper or arbitrary sleep helper. Native Playwright locators and assertions handle synchronization.

### Check your understanding

Where should “quantity three costs 1500” live? **Answer:** in the scenario's expected outcome. The detail object implements filling the quantity and adding the item; the cart object identifies cells. The arithmetic expectation belongs to the test.

# 13. Typed API contracts and client flow

## Two distinct paths

The deterministic happy path uses `ProductClient`. Its `list()` method calls an injected transport with `/api/productsList`, rejects a non-200 HTTP-like status, then runs `parseProducts`. Transport exceptions are replaced with a fixed safe error.

Live specs use `LiveHttpTransport` directly. The adapter returns HTTP status and decoded unknown JSON. The spec selects `parseProducts`, `parseBrands` or `parseRejection` and adds scenario-specific assertions. The live adapter is not a subclass of `ProductClient`, and the current live fixture does not compose the two.

```
Controlled: fixture -> ProductClient -> injected response
              -> parseProducts -> typed result
```

```
Live: fixture -> LiveHttpTransport -> fixed HTTP operation
              -> {status, unknown body}
              -> spec chooses parser -> expectations
```

## Operation and response boundaries

The operation table maps five names to fixed methods and paths. Only search sends the fixed form field `search_product: 'Blue Top'`; malformed search omits it deliberately. Automatic redirects/retries are disabled. The adapter admits and checks each redirect itself, counts it, and disposes each response in finally.

## Why this design?

Transport handles sending, status and JSON decoding; pure contracts handle shape; specs handle meaning. This makes failures easier to classify and permits in-memory adapter tests. A decoded JSON body is still unknown until validated.

The adapter checks a 1 MB limit after buffering. That limits accepted payload size but does not prevent allocation of a larger response. The roadmap records this as a deferred limitation, not a solved streaming bound.

## Check your understanding

Does `parseBrands` ensure unique IDs? **Answer:** no. It validates each entry and collection size. The live brands scenario compares Set size with array length to assert uniqueness.

# 14. UI scenario anatomy

## Controlled example: one product

In `tests/deterministic/products.spec.ts`, the UI scenario requests `catalog`, calls `open()`, then checks the item count, heading and price. Its input document contains exactly the expected synthetic fields. The browser route fulfills the reserved invalid URL locally, so navigation does not imply a contacted service.

## Live example: quantity three

The live quantity scenario requests `detail` and `cart`. `detail.addThree()` navigates to product 1, fills the spinbutton with 3, clicks Add to cart and then Continue Shopping. `cart.open()` navigates within that same session. `checkRow` verifies name, price, quantity and total, then parses displayed values for arithmetic.

Arrange: isolated context + allowlisted routes + budget  
Act: open detail -> quantity 3 -> add -> open cart  
Assert: Blue Top / 500 / 3 / 1500  
Audit: route failures + sanitized numeric evidence  
Cleanup: close the isolated context

## Read the assertion precisely

Checking one named row does not prove there are no extra rows. A successful removal checks that product 1's row disappears; it does not prove persistence after reload. Search verifies the expected card and heading, not that all returned cards match. These are recorded coverage limits.

## Why this design?

The scenario remains readable as user intent, while fixture infrastructure handles routing and evidence. Exact expected values make a provider change visible rather than silently accepting any cart total. This is also a trade-off: changing prices can fail the scenario without implying a framework defect.

## Check your understanding

If the total is wrong, should you immediately update the expectation? **Answer:** no. Inspect the intended contract and available evidence, classify the failure, and make an authorized correction only when justified. A green result is not the objective by itself.



# 15. API scenario anatomy

## Follow the negative case

`catalog.api.spec.ts` includes an unsupported-method scenario. It calls `api.send('unsupported')`, expects HTTP 200, parses a rejection body and expects provider `responseCode 405` plus the exact documented message. These assertions are intentionally not collapsed into a single success/failure flag.

```
const response = await api.send('unsupported');
expect(response.status).toBe(200);
expect(parseRejection(response.body)).toEqual({
  responseCode: 405,
  message: 'This request method is not supported.',
});
```

The other negative case sends search without its required parameter and expects provider 400 within HTTP 200. The products case checks a representative product after parsing. Search asserts that every returned name includes the fixed term; the schema already prevents an empty array from satisfying every vacuously.

## Controlled adapter evidence

`live-transport.spec.ts` injects a fake requester and dispatcher. It verifies that an evil-host redirect is rejected before a second dispatch, that a canonical redirect counts twice, that 429 stops the budget and that raw errors/non-JSON bodies become safe failures. None of those tests contacts the provider despite constructing approved-origin strings.

## Why this design?

Negative API scenarios establish how the consumer interprets rejection. Deterministic adapter tests establish handling of dangerous or malformed boundaries. They complement live observations while keeping the automatic gate repeatable.

## Check your understanding

Can HTTP 200 alone prove success? **Answer:** no. It is one transport-level fact. The provider's body and the consumer's scenario assertions establish whether the intended operation succeeded or was rejected as expected.

# 16. Network-denial preload

## Start denial before discovery

`run-tests.mjs` launches the Playwright CLI with `NODE_OPTIONS` importing `deny-network.mjs`. The preload patches selected Node entry points before tests run and sets a process-local symbol. Config and deterministic fixtures require that symbol. Workers inherit the preload through their environment.

| Surface               | Covered examples                             |
|-----------------------|----------------------------------------------|
| Sockets and protocols | TCP, TLS, HTTP, HTTPS, HTTP/2                |
| High-level APIs       | Global fetch and WebSocket                   |
| Name resolution       | DNS callbacks, promises and Resolver methods |
| Datagrams             | UDP construction, send and connect           |

`syncBuiltinESMExports()` synchronizes changed built-in exports so ESM consumers see the patched behavior. Denials use a fixed message rather than logging a rejected URL. Primitive tests use reserved `.invalid` names and expect rejection before transport.

## Browser protection is separate

Patching Node does not automatically explain all browser behavior. Chromium is configured offline with service workers blocked and DNS/background-networking restrictions. Context routes fulfill only the exact synthetic document, abort other traffic and audit denials. WebSocket routes close sockets and mark the audit.

## Why this design?

A wrapper plus config marker prevents a common mistake: starting tests through an IDE or raw CLI without the intended preload. Multiple layers address different execution paths. The child-process test demonstrates fail-closed discovery when the preload is absent.

These are safeguards for maintained code, not an OS firewall or hostile-code sandbox. A contributor who can replace scripts or execute arbitrary native code can bypass assumptions. Review and repository permissions remain necessary.

## Check your understanding

Is the marker secret proof of safety? **Answer:** no. It indicates that the expected preload ran in that process. The patched functions and routing enforce the tested behavior; the symbol alone is not a security boundary.

# 17. Browser request allowlisting

## Live browser offline, controlled HTTP effects

The live browser stays offline. `installLiveNetwork` intercepts its requests, checks `browserRequestAllowed`, then uses `route.fetch` for approved traffic and fulfills the response back to the browser. There is no permissive `route.continue()` fallback.

Browser request

- > exact host + method + resource + path + query checks
- > blocked? abort and count
- > static cache hit? fulfill and count
- > budget admission -> fetch with redirects disabled
- > validate redirect or response -> fulfill

Allowed document/XHR/fetch paths cover products, product 1 detail, cart, additions for products 1/2 and removal for product 1. Search and quantity queries are narrowly enumerated. Only first-party static CSS/scripts with approved path patterns are eligible assets; subscription scripts, images, fonts and third-party resources are blocked.

## Redirect and response checks

Every browser redirect must preserve path and query, remain on an approved host and pass the request policy. The loop allows at most three dispatch attempts. Responses with 401/403/429 or selected challenge text on navigation trigger a sticky stop. Bodies above 2 MB are rejected after buffering.

Only successful static assets are cached with body and content type. Session HTML and cookie headers are not stored in that cache. Browser session state remains in its isolated context.

## Why this design?

The controlled fetch path makes approved effects countable while browser fallback stays closed. Blocking unneeded resources reduces traffic, but it also means these are functional checks, not evidence of full visual fidelity.

## Check your understanding

Does every blocked resource fail the live scenario? **Answer:** no. Expected third-party/resource denials are counted. Unauthorized navigation and routed fetch failures set the audit failure flag. That distinction lets the site function under a deliberately limited resource policy.

# 18. Budgets, spacing, locks and time limits

## Bound each source of repeated traffic

LiveBudget belongs to the single worker and is shared across UI/API cases. Its promise queue serializes dispatch admission. It allows at most 100 sends and spaces their start times by at least one second, with an elapsed limit of five minutes. Redirects and fetched assets count; blocked requests and cache hits do not consume sends.

| Control                        | Scope                            |
|--------------------------------|----------------------------------|
| 100 sends / one-second spacing | Shared worker dispatch budget    |
| Five-minute global timeout     | Live Playwright run              |
| 315-second timeout             | Wrapper's child process          |
| Local lock directory           | Concurrent runs in this checkout |
| Manual CI concurrency group    | Live jobs in GitHub Actions      |

`stop()` is sticky. Lowering sent afterward cannot resume the same budget. A deterministic test proves that property without dispatching traffic. The queue stores a caught continuation so later calls can still reach the stopped check; it does not clear the stopped flag.

## Why one worker and no retries?

Multiple workers would create separate worker-scoped budgets. Retries could repeat external effects and hide instability. The live config uses one worker, zero retries and `maxFailures: 1`, preventing further scenarios after the first failure and avoiding a replacement worker with a fresh budget continuing the run.

The local lock is created before launching the child and removed in finally. Abrupt termination may leave it behind; inspect for an active run before removing only that stale lock. Local and CI controls are not a distributed lock, so the operator must coordinate across environments.

## Check your understanding

Does a cached stylesheet consume one of the 100 sends? **Answer:** no. It increments the cache-hit count and is fulfilled locally. A first fetch or accepted redirect does consume a send. Counters in attachments are cumulative, so do not sum every test's snapshot.

# 19. Error sanitization

## Reduce retained data at the boundary

`FrameworkError` classifies configuration, transport, contract and network failures. It does not automatically sanitize arbitrary text: callers supply fixed safe messages. `ProductClient` discards raw transport exceptions; parsers discard detailed payload validation errors.

For live execution, `sanitizeFailure` examines a message to select a broad category, replaces it and deletes stack, snippet, value, cause and `errorContext`. It runs in fixture teardown and again in reporter events. The live wrapper disables Playwright's copy prompt; raw trace, screenshot, video and browser error context are not retained by the configured lane.

```
Raw failure in maintained flow
-> safe boundary error where applicable
-> fixture sanitization
-> live reporter sanitization
-> built-in list/HTML + numeric summary
```

## What the category means

A timeout category says selector, network or environment needs triage. It does not prove which one failed. An assertion category identifies a mismatch, not a provider defect. This is deliberately less diagnostic than a raw stack or response dump.

## Why this design?

Live artifacts may otherwise retain cookies, response values or DOM content. Redacting a log line cannot sanitize an already captured trace or screenshot. The framework therefore limits capture as well as error text. That reduces evidence risk at the cost of troubleshooting detail.

The reporter keeps attachments named `traffic` and parses their JSON; it trusts the maintained fixture to supply numeric fields. It is not a general-purpose validator for arbitrary future attachments or a guarantee against malicious test output. Review remains part of the artifact policy.

## Check your understanding

Should you add raw response logging to diagnose a sanitized failure? **Answer:** not casually. Start with the scenario, safe category and traffic evidence, and obtain appropriate scope for any additional diagnostics. Never weaken the policy merely to make diagnosis easier.

## 20. Screenshots, traces and reports

### Evidence differs by lane

Deterministic failures may retain traces and screenshots because their data is synthetic. Live trace, screenshot and video capture are off. The live report uses fixed scenario metadata, status, durations and numeric traffic; a compact summary is written to `test-results/live/summary.json`.

| Output                                        | Purpose                                          |
|-----------------------------------------------|--------------------------------------------------|
| <code>playwright-report/deterministic/</code> | Controlled-run HTML report                       |
| <code>test-results/deterministic/</code>      | Synthetic failure diagnostics                    |
| <code>playwright-report/live/</code>          | Sanitized live HTML                              |
| <code>test-results/live/summary.json</code>   | Safe run result and cumulative traffic snapshots |

`npm run report` opens deterministic HTML on loopback; `npm run report:open` opens live HTML. These foreground servers stop with Ctrl+C. Opening existing evidence does not execute the suite, though its content and provenance still need review before sharing.

### Interpret the numbers carefully

Each live traffic attachment is cumulative for the shared worker. Adding every snapshot would overcount. Use the final cumulative counters to describe the run. Historical evidence records 31 sends, 170 blocked requests and 91 cache hits for each complete live suite, with separate preparation traffic documented outside those totals.

### Why this design?

Separate output paths let deterministic validation preserve live evidence. Ignoring generated reports prevents accidental publication through Git. The curated committed PNG is a reviewed exception representing the first historical live run; it is not a raw target screenshot.

Local diagnostics require manual deletion within seven days; no cleanup scheduler exists. Manual live CI uploads only sanitized report/summary with seven-day retention. Deterministic CI uploads nothing.

### Check your understanding

Why might a passing run have no screenshot? **Answer:** deterministic screenshots are failure-only, and live screenshots are disabled. Absence of a screenshot is not evidence that a test failed to execute.

## 21. CI and the automatic gate

### Same primary command, controlled inputs

The deterministic workflow runs on pull requests and pushes to main or milestone branches. It checks out code, selects the pinned Node runtime, installs locked npm packages and Chromium, then runs `npm run validate`. Afterward it checks tracked diffs and unexpected untracked files. It has read-only contents permission, pinned action commits and a 15-minute job cap.

```
Install locked dependencies + Chromium
-> doctor -> formatting -> lint -> typecheck
-> 49 deterministic tests -> support build
-> repository checks -> clean checkout checks
```

Installation downloads and GitHub operations are different from test traffic. Do not describe the whole CI job as having no network. The deterministic test lane denies outbound application traffic through its maintained controls after setup.

### Live workflow is intentionally separate

The live workflow only accepts manual `workflow_dispatch` with acknowledgment. It uses a fixed target, a concurrency group and a 12-minute job cap. Live variables are attached to the live execution step; doctor runs without them. Only sanitized report and summary are uploaded for seven days, including after failure when files exist.

### Why this design?

Public-site availability should not determine whether a routine documentation or framework change can pass its required gate. Automatic live execution would also repeat traffic without a fresh operational decision. A manual button is still not a substitute for authorization and policy review.

The study PDF command remains separate from validate. Node repository checks verify the maintained guide inventory and fingerprint; Python generation and visual inspection are explicit documentation checks. No Python package is added to npm dependencies or installed by CI in this task.

### Check your understanding

Does a local validation pass prove GitHub Actions passed? **Answer:** no. Local and remote results are separate evidence. Quote actual observed runs, and do not invent CI success for an unpushed documentation branch.

## 22. Headless and headed execution

### Visibility is one execution setting

Both Playwright configurations default to headless Chromium. Merged PR #5 added a protected headed option for live UI selection. The wrapper accepts `ui --headed`, and the package command `test:live:ui:headed` supplies that combination. It rejects headed mode with API selection, listing or no UI selection.

Reference commands below require separate live authorization and valid opt-in; they are not part of the offline study demonstration:

```
npm run test:live:ui:headed
npm run test:live:api
```

Headed UI makes browser actions visible without changing target allowlists, request budgets, retries, context isolation or capture policy. It does not authorize manual exploration outside the scenario. The same fixed UI paths still govern traffic.

### Browser-independent requests, shared browser setup

API transport uses Playwright's request context and no browser page. However, the automatic audit in `src/fixtures/live.ts` depends on context; the deterministic scenario fixture has the same shape. These scenario suites can launch headless Chromium even when their API operation is browser-independent. "No page actions" is accurate; "no browser process" is not.

### Deterministic debugging

The deterministic wrapper accepts `--debug` through `npm run test:debug`, preserving its preload. It does not accept arbitrary `--headed` or config overrides. Close Inspector when finished. Direct IDE/raw Playwright execution without the marker fails closed, so use documented wrappers.

### Why this design?

A narrow argument interface keeps interactive visibility from becoming a route around execution controls. API selection has nothing useful to demonstrate through page interactions, so the wrapper prevents a misleading headed API command.

### Check your understanding

May you use a raw Playwright command to get a visible browser? **Answer:** use the supported wrapper path. Bypassing a failing guard is not a debugging fix, and live visibility does not remove the authorization requirement.



## 23. Scripts and command wrappers

### Explain the command before running it

doctor checks exact Node version, deterministic configuration, installed Chromium and safe writable output roots. It performs no downloads. The test wrapper validates outputs again before runner cleanup. safeOutput resolves fixed repository-local roots and rejects links, including nested links.

| Command                               | Effect                                              |
|---------------------------------------|-----------------------------------------------------|
| <code>npm run doctor</code>           | Offline prerequisites and writable-output probe     |
| <code>npm run validate</code>         | Fail-fast deterministic quality sequence            |
| <code>npm run build</code>            | Safely clear dist and compile support/declarations  |
| <code>npm run check:repository</code> | Maintained-file, link, encoding and boundary checks |
| <code>npm run docs:pdf</code>         | Explicit Python workbook generation; no tests       |

The build is not an application bundle. `tsconfig.build.json` includes only `src`, while ordinary `typecheck` also checks tests and configurations. `dist` is ignored and can be reconstructed from maintained support code.

### PDF tooling contract

The Python generator reads the fixed workbook, pins and bundled Bitstream Vera prose/heading fonts. Code and bullets may use standard PDF fonts such as Courier or Helvetica; not every font is claimed embedded. Current rendering and text extraction were independently verified. It writes a temporary sibling PDF, validates page count/navigation/text, then atomically replaces the final output. Failed generation leaves the previous PDF intact. Known unsupported constructs are rejected within a deliberately limited documented subset, not a complete Markdown grammar. Handled failures remove temporary output; abrupt termination cannot guarantee cleanup. The source/build fingerprint catches stale generated evidence during Node repository checks.

### Why this design?

Wrappers establish operational boundaries before tools create processes or clean directories. Fixed paths reduce the risk of accidental broad cleanup. Separating Python tooling keeps documentation generation from expanding the npm runtime or changing the deterministic command sequence.

### Check your understanding

What should happen when an output directory is a symlink? **Answer:** output validation should reject it before cleanup. Do not weaken path checks to accommodate a convenient external report location; use the supported repository-local output structure.

## 24. How to add a UI page object

### Design exercise, not current scope

A future page-object addition begins with an approved scenario and observable risk. Decide which interactions belong together and which expectations belong in the spec. Prefer one cohesive object over a broad base class. A constructor should bind its dependency without navigating or asserting.

The existing pattern is simple:

```
constructor(private readonly page: Page) {}

async open(): Promise<void> {
  await this.page.goto('/view_cart', {
    waitUntil: 'domcontentloaded',
  });
}
```

This excerpt is from the cart pattern, not a newly implemented feature. A public method should communicate domain intent and document relevant effects. Expose a meaningful locator when the scenario needs to assert an outcome. Avoid generic “click anything” helpers that hide which operation is being performed.

### A review sequence

1. Describe the user intent and the exact observable outcome.
2. Choose accessible locators; explain a narrow fallback if a control is unlabeled.
3. Compose the object in a fixture with correct per-test lifecycle.
4. Add meaningful synthetic coverage where possible.
5. Review network requirements separately before any live use.

### Why this design?

The page object should reduce duplication of provider interactions without hiding test meaning. A new live path can exceed the current allowlist even if the object compiles. That is an operational scope change, not a reason to add `route.continue()` or expand an allowlist casually.

### Check your understanding

Would a helper that both adds a product and asserts every cart total be a good reusable page method? **Answer:** separate the action from scenario expectations. Tests may need different quantities or outcomes; forcing one assertion into the action would couple them unnecessarily.

## 25. How to add an API contract

### Start with a consumed requirement

A future API contract should validate only fields the consumer actually needs, with explicit bounds and relevant negative cases. Put pure schemas in `src/api/contracts/`, infer output types from them and accept unknown at the parse boundary. Do not import transport or fixtures into the contract module.

The current parsers follow this pattern:

```
const parsed = responseSchema.safeParse(value);
if (!parsed.success) {
  throw new FrameworkError('contract', 'Invalid product-list response.');
```

```
}
return parsed.data;
```

The fixed error avoids leaking rejected payload values. A caller receives validated fields or a classified failure; it never receives half-validated data. Additive fields are stripped by the current Zod objects, keeping consumed data stable.

### Separate three decisions

First decide the transport status expectation. Next decide the body shape and bounds. Finally decide the scenario-specific assertion, such as presence of a representative item or unique IDs. Keeping these separate prevents a schema from becoming an implicit collection of every scenario's expectations.

### Why this design?

Pure contracts are cheap to exercise deterministically. You can test a malformed ID, empty array or unexpected provider code without constructing an HTTP client. A positive case with harmless extra fields proves intended forward tolerance.

Adding a schema does not authorize a new live endpoint. A transport operation, request budget impact and target policy change need separate approved scope. This workbook introduces no new endpoint or dependency.

### Check your understanding

Should a schema accept any string price merely because a provider could change currency formatting? **Answer:** not if the scenario consumes the current rupee format. A changed consumed requirement should be reviewed explicitly rather than weakening validation to obtain a pass.

## 26. How to add deterministic scenarios

### Choose a risk worth proving

A useful deterministic test demonstrates behavior or a failure boundary, rather than repeating implementation line by line. Existing examples prove fresh builder ownership, status-before-body validation, redirect admission before a second dispatch and sticky budget exhaustion.

For a new controlled scenario, first specify the input and expected result. Use a fresh synthetic payload or minimal markup. Inject effects through the existing narrow surfaces. Do not copy live response bodies or introduce a network dependency into the automatic gate.

Risk -> controlled input -> public boundary  
-> observable result or safe failure  
-> isolation / cleanup assertion if relevant

### A practical sequence

1. Use the deterministic fixture for composed catalog/client behavior, or base Playwright test for a pure boundary.
2. Keep synthetic names and reserved invalid addresses explicit.
3. Assert the meaningful result, including a relevant rejection case.
4. Run a focused wrapper selection while developing.
5. Finish with the full deterministic validation gate.

### Why this design?

Fast focused feedback helps find a cause; the final complete gate checks interactions and repository rules. The wrapper preserves the denial preload even for a focused run. A test is not deterministic merely because its filename says so: its collaborators and effects must remain controlled.

The automatic guard audits unexpected requests after the test. If a navigation error is intentionally caught, the test still needs to account for the guard's recorded denial. Existing boundary tests exercise that behavior with an explicitly installed guard and expected audit failure.

### Check your understanding

Should you widen a browser route to make a synthetic test load a missing asset? **Answer:** add a specifically approved synthetic interception or simplify the controlled document. A permissive external fallback would invalidate the deterministic boundary.

## 27. How to add live scenarios safely

### Authorization precedes effects

This lesson describes future work only. A new live scenario needs approved purpose, target, path/method effects, lifecycle and traffic accounting. Re-check the documented policy situation before an authorized run. Missing policy text is not blanket permission; historical checks redirected to the homepage and that limitation was explicitly recorded.

| Review question                    | Why it matters                                  |
|------------------------------------|-------------------------------------------------|
| Does it create persistent data?    | Current scope allows only session cart effects. |
| Does it need a new path or method? | Existing allowlists are intentionally narrow.   |
| How many dispatches can it cause?  | Assets and redirects consume the shared budget. |
| What evidence may be retained?     | Raw live capture is disabled.                   |

### Prepare offline first

Add or update pure contract, policy and fake-transport tests under approved implementation scope. Confirm that rejected origins and operations remain rejected. Keep one worker, no retries and the current caps unless a separately reviewed requirement explicitly changes them. Never raise limits silently because a run ran out of budget.

### Why this design?

External effects deserve more scrutiny than a local assertion. A GET endpoint can still mutate a session cart, so method names alone are not a sufficient safety model. A test title does not establish the request scope either; inspect the full fixture and resource path.

After an authorized run, record pass/fail/unexecuted counts, duration, traffic and limitations. Coordinate local and CI execution because no distributed lock exists. Stop on restrictions, challenge evidence or signs of harm. A focused rerun needs a concrete correction; repeating the suite until green is not an evidence strategy.

### Check your understanding

May a new checkout scenario reuse the existing live opt-in? **Answer:** no. Checkout is outside current scope and could create persistent effects. It requires explicit new authorization and design review, not just another spec file.

## 28. Debugging and troubleshooting

### Start with the failing boundary

Run doctor first when prerequisites are uncertain. It requires exactly Node 24.21.0, an installed Chromium executable, valid deterministic settings and writable safe output roots. A different installed Node patch is an environment mismatch; do not change the pin merely to accept the machine.

| Symptom                       | First investigation                                        |
|-------------------------------|------------------------------------------------------------|
| Doctor runtime failure        | Compare active Node with <code>.node-version</code> .      |
| Missing preload error         | Use the documented npm wrapper.                            |
| Contract rejection            | Examine controlled input and consumed schema fields.       |
| Browser request audit failure | Identify the scenario's unexpected resource or navigation. |
| Live sanitized timeout        | Triage selector/network/environment from safe evidence.    |
| Stale PDF fingerprint         | Regenerate canonical source with pinned tooling.           |

### Separate environment failures from framework failures

An OS process-spawn restriction can prevent Playwright from launching before any test runs. A missing binary can have the same practical result. Neither is evidence that the provider failed. Report the command, actual failure stage and whether tests executed.

For deterministic behavior, reproduce with a focused wrapper selection after a justified correction, then run the complete gate. Use `npm run test:debug` for Inspector without dropping the preload. Do not set live variables to diagnose an offline failure.

### Why this design?

The framework preserves enough boundary information to begin diagnosis without printing all environment values or provider bodies. Safe categories are intentionally broad. When evidence is insufficient, report “untriaged” instead of guessing a root cause.

PDF troubleshooting is separate: activate the isolated Python environment, install exact pins, run `npm run docs:pdf`, inspect rendered pages and check hashes. The script rejects known unsupported Markdown constructs and oversized code. It implements a documented subset, not a complete Markdown grammar.

### Check your understanding

If a live lock exists, should you delete the whole results directory? **Answer:** no. First verify no run is active, then handle only the stale lock within authorized scope. Broad cleanup can erase evidence or interfere with another process.

## 29. Framework limitations and evidence

### State the supported claims precisely

The baseline automatic suite has 49 deterministic cases and a single Chromium project. The separate live suite has nine scenarios. Two complete live suites passed on 2026-09-24; only the first used supported Node 24.21.0. The second, initiated by Stefan, used Node 24.13.0 and is owner verification rather than supported-runtime or clean-install evidence.

The canonical live run took 33,471.642 ms and recorded 31 sends, 170 blocked requests and 91 cache hits. Preparation traffic is documented separately. These are historical observations, not a guarantee that the public site will pass now.

### Recorded and newly documented limitations

- Search UI does not prove every returned card matches; cart coverage omits exact whole-cart row count and reload persistence.
- Some exported types/constants still lack concise TSDoc; this study task does not silently fix them.
- API and browser response limits apply after buffering, and HTTP 303 handling differs between the layers.
- Newly documented during the study-guide review: API requests use no Playwright page, but shared automatic fixtures may still create a browser context. This potential future optimization is not an accepted historical N-1–N-5 disposition.
- Network controls cover maintained code and tested primitives, not hostile native code or a distributed execution environment.
- Blocked resources limit visual fidelity; Firefox/WebKit and comprehensive accessibility remain deferred.

### Why this design?

Honest limits help an interviewer assess judgment. A portfolio framework need not solve every problem, but it should separate implemented evidence from intended future work. The roadmap contains accepted non-blocking findings; mentioning one and describing an appropriate next step is stronger than pretending complete coverage.

No production users, business impact, checkout/payment coverage or open-source license grant are established. MIT is intended, but no license file exists. Release and deployment are not part of this study package.

### Check your understanding

Can “portfolio-ready” be translated into “production-proven”? **Answer:** no. Portfolio readiness describes presentation and demonstrated engineering work, not evidence of real production operation or comprehensive correctness.

## 30. Interview exercises and answers

### Explain the project in thirty seconds

“This portfolio framework separates repeatable UI/API framework checks from authorized live observations. It uses Playwright, strict TypeScript, injected fixtures and Zod contracts. Automatic validation runs 49 deterministic tests with network denial. A separate nine-scenario live lane has exact targets, bounded traffic and sanitized reports. It demonstrates engineering practice, not production-user impact.”

### Exercise 1: trace a UI assertion

Start at `npm test`, explain the preload and config marker, then the fixture, intercepted document, page object and assertion. **Answer checkpoint:** say that the invalid-host document is fulfilled locally; do not describe it as a running application server. Mention teardown auditing and synthetic failure capture.

### Exercise 2: explain HTTP 200 with provider 405

**Answer:** transport and application-layer statuses differ. The unsupported-method spec deliberately checks HTTP 200, parses the rejection body, then checks provider 405 and the expected safe message. A single status assertion would miss the consumer contract.

### Exercise 3: challenge the “browser-free API” claim

**Answer:** the request uses no page, but the shared automatic audit requires a browser context. That fixture graph creates browser overhead. A future fixture split could remove it, but the current documentation must describe the implementation honestly.

### Exercise 4: defend one trade-off

**Answer:** no retries and one worker reduce throughput but keep external effects and budget ownership understandable. Live fail-fast avoids continuing with a replacement worker and fresh budget. The same choice does not prove the absence of flakiness; it makes failures visible.

### Five-minute rehearsal

Show the README and architecture, trace the deterministic spec and fixture, run the two read-only scenarios, then discuss the report and a recorded limitation. Prepare runtime/dependencies first. Do not turn a failed offline demo into an unplanned live run.

### Final self-check

Can you explain where data becomes trusted, where effects occur, who owns cleanup and what the evidence cannot prove? If so, you understand the architecture beyond its file names. For detailed questions use `docs/code-walkthrough.md`; for timed answers and the presentation checklist use `docs/interview-guide.md`.